

```

import os
import re

from google import genai
from google.genai import types
import matplotlib.pyplot as plt
import numpy as np
from pydantic import BaseModel, Field

def get_gemini_client():
    """Initializes the Gemini GenAI client using the embedded API key."""
    # Your provided Gemini API Key
    api_key = "AQ.Ab8RN6Imn0wIhoQtaNm0vq4eerpH3ofjN0n2STcZ3c-B7eUhjQ"

    # Clean system flags to avoid OAuth enterprise fallback loops
    for var in ["GOOGLE_APPLICATION_CREDENTIALS", "CLOUD_SDK_CREDENTIALS"]:
        if var in os.environ:
            del os.environ[var]

    return genai.Client(api_key=api_key)

class LinearSystemSchema(BaseModel):
    variable_labels: list[str] = Field(
        description="List of text labels for each variable discovered in the text problem up to 4 dimensions."
    )
    matrix_a: list[list[float]] = Field(
        description="2D array matrix matching the system variable coefficients."
    )
    vector_b: list[float] = Field(
        description="1D array containing the structural constant sum variables."
    )
    explanation_of_equations: list[str] = Field(
        description="Brief text breakdown details of how equations were formulated."
    )

def universal_llm_parser(problem_statement: str) -> tuple[np.ndarray, np.ndarray, list[str]]:
    """Translates unstructured word problem text into structured matrix arrays using Chat workflows."""
    client = get_gemini_client()

    prompt = f"""
    You are an expert mathematical translator. Parse the following real-world word problem into a system of linear equations.
    """

```

```

    Problem: "{problem_statement}"
    Instructions:
    1. Identify all unknown variables (up to 4). Give them clear, meaningful labels
    (e.g., 'x_3percent', 'apples', 'age_john').
    2. Extract the system of linear equations from the text constraints.
    3. Convert these equations into standard matrix forms:
        - Matrix A: a 2D array representing coefficients of the variables.
        - Vector B: a 1D array representing the constants/totals on the right side
    of the equations.
    """
    print(
        "\n[AI Parsing Engine] Analysing text semantics and extracting linear
    constraints..."
    )

    # REFACTOR: Using Chat session to conform to SDK optimization standards
    chat = client.chats.create(
        model="gemini-3.8-flash",
        config=types.GenerateContentConfig(
            response_mime_type="application/json",
            response_schema=LinearSystemSchema,
            temperature=0.1,
        )
    )

    response = chat.send_message(prompt)
    parsed_data = response.parsed

    print("\n[AI Explanation of Formulated Equations]:")
    for step in parsed_data.explanation_of_equations:
        print(f" -> {step}")

    return (
        np.array(parsed_data.matrix_a, dtype=float),
        np.array(parsed_data.vector_b, dtype=float),
        parsed_data.variable_labels,
    )

def solve_system_via_row_echelon(A, B):
    """Converts augmented matrix to Row Echelon Form and resolves the values."""
    if A.size == 0 or B.size == 0:
        return "Invalid Problem Structuring", None

    M = np.hstack([A, B.reshape(-1, 1)])
    rows, cols = M.shape
    num_vars = cols - 1
    tol = 1e-7

    pivot_row = 0

```

```

for col in range(num_vars):
    if pivot_row >= rows:
        break
    max_row = np.argmax(abs(M[pivot_row:, col])) + pivot_row
    if np.isclose(M[max_row, col], 0, atol=tol):
        continue
    M[[pivot_row, max_row]] = M[[max_row, pivot_row]]
    for r in range(pivot_row + 1, rows):
        factor = M[r, col] / M[pivot_row, col]
        M[r] -= factor * M[pivot_row]
    pivot_row += 1

rank_A = 0
rank_augmented = 0
for r in range(rows):
    if not np.allclose(M[r, :num_vars], 0, atol=tol):
        rank_A += 1
    if not np.allclose(M[r, :], 0, atol=tol):
        rank_augmented += 1

print(
    f"\n[Matrix Core] Rank of A: {rank_A}, Rank of Augmented Matrix:
{rank_augmented}"
)

if rank_A < rank_augmented:
    return "No Solution (Inconsistent system)", None
elif rank_A < num_vars:
    return "Infinite Solutions (Underdetermined system)", None

solutions = np.zeros(num_vars)
for i in range(num_vars - 1, -1, -1):
    if np.isclose(M[i, i], 0, atol=tol):
        return "No Solution", None
    solutions[i] = (
        M[i, -1] - np.dot(M[i, i + 1 : num_vars], solutions[i + 1 : num_vars])
    ) / M[i, i]

return "Unique Solution", solutions

def plot_universal_distribution(labels, values):
    """Charts the computed variables."""
    plt.figure(figsize=(9, 5))
    colors = ["#4C72B0", "#55A868", "#C44E52", "#CCB974"][: len(labels)]
    bars = plt.bar(
        labels, values, color=colors, edgecolor="grey", alpha=0.85
    )
    plt.ylabel("Computed Quantitative Output")
    plt.title("Resolved System Variables Allocation Profile")

```

```

plt.grid(axis="y", linestyle="--", alpha=0.4)

max_val = max(abs(values)) if len(values) > 0 else 1
offset = (max_val * 0.02) if max_val > 0 else 0.1
for bar in bars:
    yval = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2.0,
        yval + offset,
        f"{yval:,.2f}",
        ha="center",
        va="bottom",
        weight="bold",
    )
plt.show()

if __name__ == "__main__":
    print("=====")
    print("                UNIVERSAL REAL-WORLD WORD PROBLEM SOLVER                ")
    print("=====")

    user_problem = input("Enter your word problem statement text below:\n>
").strip()

    if not user_problem:
        print("[Error] Empty statement. Terminating execution.")
    else:
        try:
            matrix_A, vector_B, variable_names = universal_llm_parser(
                user_problem
            )
            print("\nGenerated Coefficient Matrix (A):\n", matrix_A)
            print("Generated Constant Vector (B):\n", vector_B)

            status, results = solve_system_via_row_echelon(matrix_A, vector_B)
            print(f"\n[Matrix Resolution Status]: {status}")

            if status == "Unique Solution":
                print(
                    "\n===== FINAL SOLVED VALUES ====="
                )
                for var, val in zip(variable_names, results):
                    print(f" -> {var} = {val:,.2f}")
                print(
                    "=====
                )
                plot_universal_distribution(variable_names, results)
            else:
                print(

```

```
                f"[Execution Warning] Stop: The system returned status:
'{status}'"
            )
    except Exception as e:
        print(f"\n[System Failure Exception Details]: {e}")
```